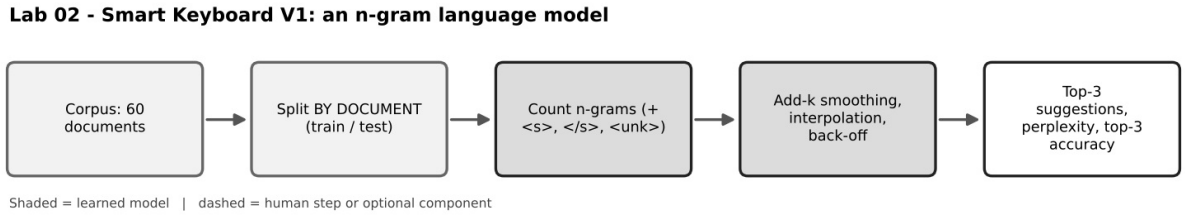
Lab 02 — Smart Keyboard V1

Mission: build the engine behind phone-keyboard suggestions: an **n-gram language model** that predicts the next word, then measure it with **perplexity** and **top-3 accuracy**.

Level	Beginner–intermediate
Duration	30-minute track or 60-minute track
Data	1,200 synthetic sentences in 60 documents (≈10,000 tokens, 61 word types) generated by a fixed-seed grammar — included
Libraries	Python standard library, matplotlib, Streamlit



Lab 02 pipeline

1. Why this lab, and its final goal

A language model assigns a probability to the next word given what was typed. It is the oldest and simplest idea in modern NLP — and the same quantity (`P(next word | history)`) that large neural models optimise today.

Final goal: train unigram, bigram and trigram models, **smooth** them so that unseen word sequences do not get probability zero, and compare them with **perplexity** and **top-3 accuracy** on held-out documents.

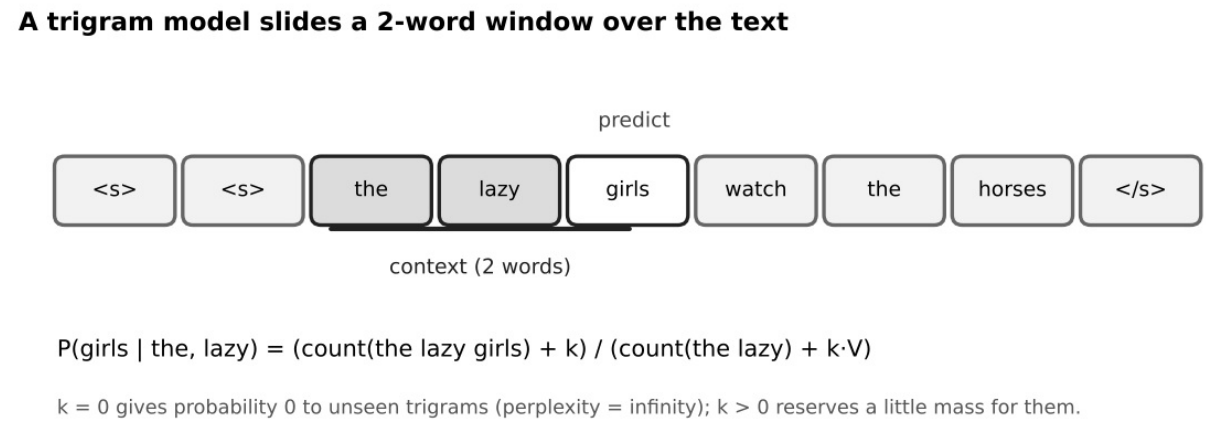
2. What you will be able to do afterwards

1. Compute `P(w | context)` from counts and explain maximum-likelihood estimation.
2. Explain *why* unsmoothed models fail (zero probabilities → infinite perplexity) and fix it with add-k smoothing, interpolation and back-off.
3. Compute and interpret **perplexity** and **top-k accuracy**.
4. Split data **by document**, not by sentence, and explain the leak that this avoids.
5. Diagnose model errors by listing contexts that were never seen in training.

3. Skills, techniques and algorithms used

Category	Item	Where it appears
Language modelling	n-gram counting with `<s>`, `</s>`, `<unk>` symbols	`NGramModel.__init__`
Language modelling	Maximum-likelihood estimate and **add-k (Laplace) smoothing**	`NGramModel.prob`
Language modelling	**Linear interpolation** of trigram / bigram / unigram	`InterpolatedModel`
Language modelling	**Back-off** to a lower order when the context is unseen	`backoff_top_k`
Evaluation	**Perplexity** = exp(average negative log-probability)	`NGramModel.perplexity`
Evaluation	Top-k accuracy of next-word suggestions	`top_k_accuracy`
Methodology	Train/test split **by document**; hyper-parameter sweep; out-of-vocabulary handling	`split_by_document`, `evaluate.py`
Engineering	Streamlit app with live suggestions	`app.py`

4. The concept in one picture



A trigram model predicts the next word from the previous two.

Formulas

- Trigram probability with add-k smoothing: `P(w | u v) = (count(u v w) + k) / (count(u v) + k·V)`, `V` = vocabulary size.
- Perplexity: `PP = exp( − (1/N) Σ log P(w_i | context_i) )`. Intuition: *the model is as unsure as if it had to pick uniformly among `PP` words at every step.* Lower is better; a uniform guess over this vocabulary would be about 61.
- Interpolation: `P = 0.6·P₃ + 0.3·P₂ + 0.1·P₁` (the weights are fixed here).

5. Setup (about 3 minutes)

```
# 1. open a terminal in the folder of the lab
cd 02_smart_keyboard_v1

# 2. create an isolated Python environment (Python 3.10+)
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

# 3. install the packages
pip install -r requirements.txt
#   = matplotlib streamlit

# 4. generate the dataset (fixed seed: everybody gets identical data)
python data/make_data.py

# 5. run the full evaluation (prints every number used in this guide)
python -m src.evaluate

# 6. launch the small web app
streamlit run app.py
```

6. Two tracks

Time	30-minute track	60-minute track (adds)
0–5	Setup, look at `data/corpus.csv`	Predict which model (1-, 2- or 3-gram) will win
5–15	Steps 1–3 (split, count, probability)	Steps 1–4
15–25	Step 5: run the evaluation and read the table	Step 5 + Step 6 (smoothing sweep)
25–30	Short report	Step 7 (app) + Step 8 (failures) + full report

7. Step by step

Step 1 — The data and the split (3 min)

The corpus is a CSV with `doc_id` and `sentence`. We split **by document**: the last 20 % of documents are the test set (960 train sentences / 240 test sentences).

```
def split_by_document(docs, test_fraction=0.2):
    """Fixed, reproducible split BY DOCUMENT (the last 20 % of documents are the test set)."""
    ids = sorted(docs)
    cut = int(len(ids) * (1 - test_fraction))
    train = [s for i in ids[:cut] for s in docs[i]]
    test = [s for i in ids[cut:] for s in docs[i]]
    return train, test
```

> **Why not split by sentence?** Sentences of one document share topic words. Putting some in train and others in test lets the model "see" the test vocabulary and makes scores look better than they are.

Step 2 — Count n-grams (5 min)

```
class NGramModel:
    def __init__(self, sentences, n=3, k=1.0):
        self.n, self.k = n, k
        self.vocab = {w for s in sentences for w in s} | {EOS, UNK}   # BOS is never predicted
        self.V = len(self.vocab)
        self.ctx_counts = defaultdict(Counter)       # context tuple -> Counter(next word)
        for s in sentences:
            padded = [BOS] * (n - 1) + s + [EOS]
            for i in range(n - 1, len(padded)):
                self.ctx_counts[tuple(padded[i - n + 1:i])][padded[i]] += 1
        self.ctx_totals = {c: sum(cnt.values()) for c, cnt in self.ctx_counts.items()}

    # -- helpers
    def clean(self, words):
        """Map out-of-vocabulary words to <unk> (so the model never crashes on new words)."""
        return [w if w in self.vocab else UNK for w in words]

    def context(self, words):
        words = [BOS] * (self.n - 1) + self.clean(words)
        return tuple(words[-(self.n - 1):]) if self.n > 1 else ()

    def prob(self, ctx, word):
        counts = self.ctx_counts.get(ctx)
        total = self.ctx_totals.get(ctx, 0)
        num = (counts[word] if counts else 0) + self.k
        den = total + self.k * self.V
        return num / den if den > 0 else 0.0         # k = 0 and unseen context -> probability 0

    def next_word_distribution(self, words):
        ctx = self.context(words)
        return {w: self.prob(ctx, w) for w in self.vocab if w != UNK}

    def top_k(self, words, k=3):
        dist = self.next_word_distribution(words)
        return sorted(dist.items(), key=lambda x: (-x[1], x[0]))[:k]

    def seen_context(self, words):
        return self.context(words) in self.ctx_counts

    # -- evaluation
    def perplexity(self, sentences):
        """exp( - mean log P(token) ) over every token of every sentence (EOS included).
        Returns (perplexity, share_of_zero_probability_tokens)."""
        log_sum, n_tok, zeros = 0.0, 0, 0
        for s in sentences:
            words = self.clean(s)
            padded = [BOS] * (self.n - 1) + words + [EOS]
            for i in range(self.n - 1, len(padded)):
                p = self.prob(tuple(padded[i - self.n + 1:i]), padded[i])
                n_tok += 1
                if p <= 0:
                    zeros += 1
                else:
                    log_sum += math.log(p)
        if zeros:
            return float("inf"), zeros / n_tok
        return math.exp(-log_sum / n_tok), 0.0
```

Read `__init__`: each sentence is padded with `n−1` start symbols `<s>` and an end symbol `</s>`; every `(context → next word)` pair is counted. `clean()` maps unseen words to `<unk>` so the model never crashes on new vocabulary.

Step 3 — Probability with smoothing (5 min)

`prob(ctx, word)` is the formula of section 4. Try it:

```
from src.pipeline import load_documents, split_by_document, NGramModel
train, test = split_by_document(load_documents())
```

```
m = NGramModel(train, n=3, k=0.1)
print(m.top_k(["the", "horses", "eat", "the"], 3))   # [('bread', 0.168), ('apples', 0.15), ('soup', 0.144)]
```

Only the last two words (`eat the`) are used by a trigram model; the foods it suggests come from the training counts.

With `k = 0` an unseen context gives probability 0 — the model is **brittle**. With `k = 1` it is too generous (it gives a lot of mass to impossible sequences). That is why we sweep `k` in Step 6.

Step 4 — Evaluate: perplexity and top-3 accuracy (5 min)

```
def top_k_accuracy(model, sentences, k=3):
    """Share of positions where the true next word is among the k suggestions."""
    hits = total = 0
    failures = []
    for s in sentences:
        for i, true_w in enumerate(s + [EOS]):
            guess = [w for w, _ in model.top_k(s[:i], k)]
            truth = true_w if true_w in model.vocab else UNK
            total += 1
            if truth in guess:
                hits += 1
            elif len(failures) < 200:
                failures.append((" ".join(s[:i]) or "<start>", true_w, guess))
    return hits / total, failures
```

For each position of each test sentence we ask:

is the true next word among the 3 suggestions?

Step 5 — Run the evaluation (5 min)

```
python -m src.evaluate
```

```
Split BY DOCUMENT (fixed): train = 960 sentences / 8098 tokens, test = 240 sentences / 2070 tokens

model                                perplexity   zero-prob tokens   top-3 accuracy
unigram (k=1)                             24.96              0.0%            48.0%
bigram (k=1)                               6.43              0.0%            60.9%
trigram (k=1)                              8.30              0.0%            64.1%
trigram (k=0.1)                            5.06              0.0%            64.1%
trigram unsmoothed (k=0)                    inf              4.6%            64.1%
interpolated 3/2/1-gram (k=0.1)            5.15              0.0%            64.7%

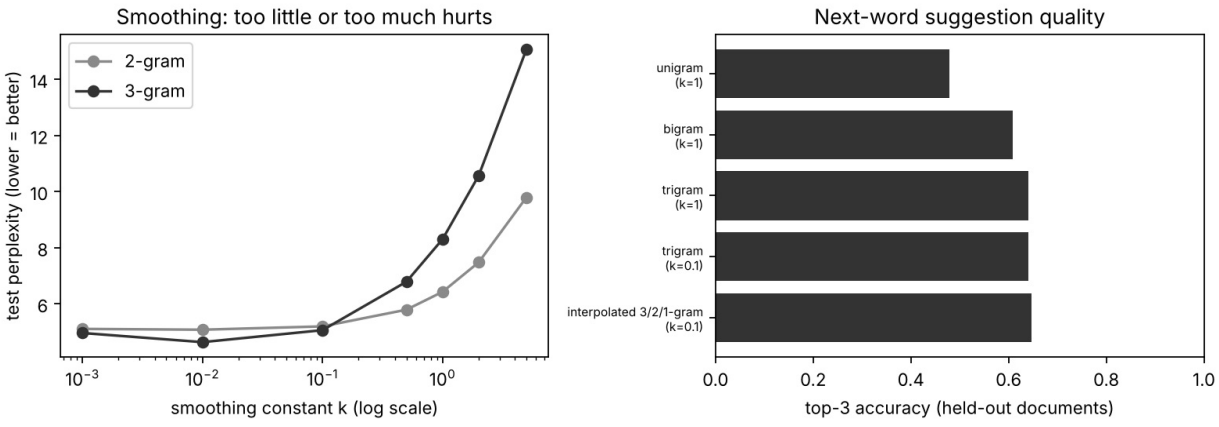
Hard back-off (3->2->1-gram, k=0.1) top-3 accuracy: 64.2%

Perplexity vs k:
  2-gram: k=0.001: 5.11, k=0.01: 5.08, k=0.1: 5.20, k=0.5: 5.79, k=1: 6.43, k=2: 7.48, k=5: 9.80
  3-gram: k=0.001: 4.96, k=0.01: 4.63, k=0.1: 5.06, k=0.5: 6.79, k=1: 8.30, k=2: 10.57, k=5: 15.07

Trigram (k=0.1) top-3 misses (context -> true word | suggestions):
  'the' -> 'foxes' | ['doctors', 'teachers', 'girls']
  'the foxes eat the' -> 'cakes' | ['bread', 'apples', 'soup']
  'the' -> 'boys' | ['doctors', 'teachers', 'girls']
  'the boys' -> 'sell' | ['watch', 'buy', 'bring']

Test positions whose trigram context never appears in training: 9
  examples: ['brave foxes', 'brave girls', 'happy bread', 'hungry key', 'lazy soup']

Saved results/metrics.json, results/failures.txt, results/ngram_results.png
```



Perplexity per model and perplexity as a function of k (results/ngram_results.png)

How to read it

- Unigram perplexity ≈ 25: ignoring context is poor.
- **Trigram, k = 0.1 → 5.06**: much better, and top-3 accuracy of **64 %** — on a 61-word vocabulary with random-looking object choices (`eat the bread / apples / soup`), this is close to the ceiling of the grammar.
- **Unsmoothed trigram → ∞** because 4.6 % of the test tokens have probability exactly 0: **one** impossible token makes perplexity infinite.
- Interpolation (5.15) is slightly *worse* than the tuned trigram on perplexity but marginally better on top-3 accuracy (64.7 %): different metrics reward different things.

Step 6 — Smoothing sweep (60-minute track)

Look at the `Perplexity vs k` lines: for the trigram the best value among those tried is near `k = 0.01` (4.63). **Warning:** this sweep uses the *test* set, which is acceptable for teaching but not for a real study — tune on a separate validation split and report once on test (Lab 09 does it properly).

Step 7 — The app (60-minute track)

```
streamlit run app.py
```

Type `the horses eat the` and watch the three suggestions change as you type. Try a word that is not in the corpus: the app should say it is unknown (and continue with `<unk>`), never crash.

Step 8 — Failure analysis (mandatory)

The evaluation prints test positions where the correct word is not in the top 3, and the contexts never seen in training (9 positions, e.g. `brave foxes`, `happy bread`).

Failure type	Example	Explanation
Ambiguity at sentence start	`the` → `foxes`	After `the` almost any noun is possible; 3 guesses cannot cover 20 nouns
Unseen context	`lazy soup`	Combination never produced by the grammar in training → back-off to a lower order
Semantic choice	`eat the` → `cakes`	Several foods are equally likely; the n-gram has no notion of *meaning* (see Lab 07)

8. Deliverables

1. Output of `python -m src.evaluate`.
2. Answer in 5 lines: *which model is best, by how much, and why smoothing is needed.*
3. (60-min) Your own `k` sweep conclusion and one failure case with a proposed fix.

9. Grading rubric (10 points)

Criterion	Points
Correct split by document, explained	2
Working models and results table	2
Correct explanation of perplexity and of the infinite value	2
Failure analysis with ≥ 2 cases	2
Smoothing sweep or app demo	1
Clarity	1

10. Common pitfalls

- Computing perplexity with a different vocabulary for each model (not comparable).
- Letting test words into the vocabulary (leak).

- Forgetting `</s>` in the probability: the model must also predict the *end* of a sentence.

11. Reflection questions

1. Why does perplexity get *worse* between bigram (k=1) and trigram (k=1) but better for the trigram with k=0.1?
2. Would a 5-gram help on this 960-sentence corpus? What does data sparsity imply?
3. Why is perplexity not comparable between a word-level and a character-level model?
4. Suggest a way to use the app *without* sending typed text to a server (privacy).

12. Extensions

Add Kneser-Ney smoothing, or train on your own text (a WhatsApp export is **personal data** — only with your own consent and never share it).

13. Limits and ethics

- **Data are synthetic** and grammatical by construction; real text is far harder (perplexities of 100+ are normal).